

ZONKE REDEVELOPMENT PROJECT

TEAM 5 — TESTING & QUALITY

Field	Details
System Developers	Benedict Monsemvula, Maxwell Zandanele
Role	QA / Testers
Main Responsibility	Test the Zonke application continuously as it is built and provide evidence of defects, fixes and retesting.
Tools / Technologies	Browser; GitHub; spreadsheet or document for test cases; screenshots; browser developer tools; Postman or similar tool when needed.

1. What You Must Do

Task	Instruction
1. Prepare test cases	Create practical test cases based on the agreed requirements and user flows.
2. Test early	Start testing as soon as a feature or screen is available. Do not wait for the whole application to be completed.
3. Functional testing	Check whether features perform the required action and whether expected results are produced.
4. Responsive testing	Check important screens at different screen sizes and identify layout or interaction problems.
5. Usability checks	Check whether users can understand navigation, forms, labels, messages and basic workflows.
6. Accessibility checks	Check basic accessibility issues such as readable text, labels, keyboard access where relevant, contrast and clear feedback.
7. Record bugs	Document the steps to reproduce a defect, expected result, actual result, severity/priority and evidence.
8. Retest fixes	When a defect is reported as fixed, test it again and record whether the fix works.
9. Report quality	Keep the team informed about important defects, repeated failures and features that are ready for wider testing.

2. Work in This Order

Step	What to do
1. Prepare	Create test cases from requirements and user flows.
2. Test	Test available functionality as soon as it becomes available.
3. Record	Document bugs clearly with evidence.
4. Retest	Verify fixes and record the result.
5. Review	Report outstanding issues and quality concerns.

3. Computer and Project Setup

Item	Instruction
Computer	Use a reliable computer with enough storage for the development tools, project files and Git repository. Keep the operating system and browser updated.
Code editor	Install VS Code or another editor you already understand. Make sure extensions required for your chosen technology are installed.
Git	Install Git and configure your name and email. Confirm that you can clone, pull, commit and push to the project repository.
GitHub	Use the shared Zonke repository. Work on your own branch where the team workflow requires it, pull before starting work, and commit regularly.
Browser	Use a modern browser such as Chrome, Edge or Firefox. Keep developer tools available for inspecting and testing the application.

Item	Instruction
Project folder	Create a clear local project folder. Do not keep the main project only on the desktop or in random folders.
Dependencies	Install only the packages, libraries, frameworks or tools actually required by the chosen technology. Record important setup steps so another System Developer can reproduce the environment.
Technology choice	Use technologies that the team understands and can explain. Do not choose a framework simply because it is popular or because AI suggested it.
Current Zonke site	Use the existing Zonke website as a reference for understanding the business, content and user journeys. The new application is being redeveloped from scratch.

4. Team-Specific Setup / Guidance

Area	Instruction
Project repository	All five teams contribute to the same Zonke development project. Do not create separate unrelated versions of the application.
Team communication	Keep the other teams informed about decisions, dependencies, changes to requirements and anything that may affect their work.
Use of AI	AI can help with explanations, ideas and troubleshooting, but it must not replace your understanding. Check, test and adapt any AI-generated material before using it.
Do not change business decisions	Do not invent or change the business model, target market, payment approach, user-data requirements, domain or hosting decisions without confirmation from the project/CTO direction.
Testing environment	Know which local URL, branch/build or API environment you are testing. Record the version or commit where useful so results can be reproduced.
Evidence	Use screenshots, screen recordings where appropriate, console output, API responses or other useful evidence. Do not collect screenshots that do not prove anything.
Bug reports	A good bug report should allow another System Developer to reproduce the problem without guessing.
Continuous testing	If Team 3 has a homepage on Tuesday, test it Tuesday. If Team 4 has an API on Wednesday, test it Wednesday. After a fix, retest the same issue.

5. By Friday

Required Output	What should be available
Test-case list	A practical list of tests for the available prototype functionality.
Completed tests	Evidence that available features have already been tested.
Bug reports	Documented bugs with clear reproduction steps and evidence.
Retest results	Results showing whether reported fixes actually work.
Outstanding issues	A clear list of important unresolved issues that development teams need to address.

6. Working Rules

Rule	Instruction
GitHub	Commit regularly. Commit messages must describe the work completed. Each System Developer must be able to show their own contribution.
Communication	Tell the other teams when work is ready, blocked or dependent on another team's work.
Evidence	Keep useful evidence such as screenshots, designs, code commits, test results, diagrams or API results. Evidence should prove what was actually done.
Understanding	Do not copy and paste work from AI without understanding it. You must be able to explain your code, design decisions, tests and recommendations.
Professional practice	Keep files organised, use clear names, avoid unnecessary duplication, and ask for help when you are genuinely blocked.

Rule	Instruction
Build from scratch	The new Zonke application is a fresh redevelopment. The existing website is a reference point, not a codebase that must be copied.